

Deployment Guide

Product: TracSOpro
Last updated: June 2026

This guide covers production deployment for the backend API and mobile app release builds.

Prerequisites

Requirement	Notes
Node.js 20+	Backend and mobile build tooling
PostgreSQL 16+	Production database
DigitalOcean droplet	Current production host (or equivalent VPS)
PM2	Process manager on the server
Firebase project	FCM push notifications
Stripe account	Subscriptions and billing
SMTP provider	OTP and password reset emails
Android keystore	Release APK signing (not debug)

Backend deployment

Environment setup

Create `backend/.env` on the server. See [ENVIRONMENT_VARIABLES.md](#) for the full list.

Minimum production variables:

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://user:pass@host:5432/tracsopro
JWT_SECRET=<strong-32+-char-secret>
JWT_EXPIRES_IN=30m
REFRESH_TOKEN_EXPIRES_IN=7d
CORS_ORIGIN=https://api.tracsopro.com
SMTP_HOST=...
SMTP_PORT=587
SMTP_USER=...
SMTP_PASS=...
SMTP_FROM=noreply@tracsopro.com
SMTP_DEV_BYPASS=false
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
FIREBASE_SERVICE_ACCOUNT_PATH=./keys/firebase-service-account.json
```

Place the Firebase service account JSON at the path specified. Never commit this file to git.

Manual deployment

```
# On the server
cd /root/backend
npm ci
npx prisma generate
npx prisma migrate deploy
pm2 restart guard-tracking-api || pm2 start npm --name "guard-tracking-api" --
start
```

Verify: `curl https://api.tracsopro.com/api/health`

CI/CD (GitHub Actions)

Pushing to `main` with changes under `backend/**` triggers `.github/workflows/deploy-backend.yml`:

1. SCP backend files to the DigitalOcean droplet (`/root/backend`).
2. SSH in and run `npm ci`, `prisma generate`, `prisma migrate deploy`.
3. Restart PM2 process `guard-tracking-api`.

Required GitHub secrets:

- `DROPLET_IP` — server IP address
- `DROPLET_SSH_KEY` — SSH private key

Manual trigger: GitHub Actions → Deploy Backend to DigitalOcean → Run workflow.

Database migrations

Command	When to use
<code>npx prisma migrate dev</code>	Local development only
<code>npx prisma migrate deploy</code>	Production — applies pending migrations
<code>npx prisma db push</code>	Dev prototyping only — never in production

Migration files: `backend/prisma/migrations/`

PM2 process

```
pm2 start npm --name "guard-tracking-api" -- start
pm2 save
pm2 startup    # Enable auto-restart on reboot
```

The `start` script runs `server-db.ts` (production entry point with WebSocket, Firebase, and background jobs).

Stripe webhook

Register the webhook endpoint in the Stripe dashboard:

```
https://api.tracsopro.com/api/payments/webhook
```

Events to subscribe: `checkout.session.completed`, `invoice.paid`, `customer.subscription.updated`, `customer.subscription.deleted`.

Set `STRIPE_WEBHOOK_SECRET` from the Stripe dashboard webhook signing secret.

CI tests

`.github/workflows/backend-tests.yml` runs on push/PR:

- Spins up PostgreSQL 16 service container.
- Runs `prisma db push` and `npm run test:tenant` (tenant isolation tests).

`.github/workflows/security.yml` runs Gitleaks and a custom secrets scanner.

Mobile app deployment

Pre-release checklist

Before any production or store build, confirm:

- ☐ `USE_LOCAL_LAN_RELEASE` is `false` in `apiConfig.ts`
- ☐ Android release keystore is configured (not debug signing)
- ☐ Production `.env` is set on the server (see [ENVIRONMENT_VARIABLES.md](#))
- ☐ Database migrations applied: `npx prisma migrate deploy`

- ☐ Stripe webhook and Firebase credentials are production values
- ☐ Smoke test passed for all four roles against production API

API configuration

Edit `GuardTrackingApp/src/config/apiConfig.ts`:

```
const USE_LOCAL_LAN_RELEASE = false; // MUST be false for production
const PRODUCTION_API_URL = 'https://api.tracsopro.com/api';
const PRODUCTION_WS_URL = 'https://api.tracsopro.com';
```

Android release build

1. **Create a release keystore** (one-time):

```
keytool -genkeypair -v -storetype PKCS12 -keystore release.keystore \
  -alias tracsopro -keyalg RSA -keysize 2048 -validity 10000
```

2. **Configure signing** — copy `android/keystore.properties.example` to `android/keystore.properties`:

```
storeFile=../release.keystore
storePassword=<password>
keyAlias=tracsopro
keyPassword=<password>
```

3. **Disable cleartext traffic** — ensure `android/app/src/main/res/xml/network_security_config.xml` does not allow HTTP in release.

4. **Build:**

```
cd GuardTrackingApp
npm run android:release          # Gradle assembleRelease
# or
npm run android:release:local    # PowerShell helper script
```

Output APK: `android/app/build/outputs/apk/release/app-release.apk`

iOS release build

1. Open `ios/GuardTrackingApp.xcworkspace` in Xcode.
2. Set signing team and bundle identifier.
3. Configure Firebase (`GoogleService-Info.plist`).
4. Archive and upload to App Store Connect.

Firestore setup

Platform	File
Android	<code>android/app/google-services.json</code>
iOS	<code>ios/GoogleService-Info.plist</code>
Backend	<code>backend/keys/firebase-service-account.json</code>

Push notification validation

After deploying backend + mobile:

1. Log in on a physical device.
2. Confirm device token registers (GET `/api/notifications/push-status`).
3. Send test push (POST `/api/notifications/test-push`).
4. Trigger an emergency alert and verify delivery.

Deployment order

Recommended sequence for a new release:

1. Apply database migrations (prisma migrate deploy)
2. Deploy backend (CI or manual)
3. Verify `/api/health` and `/api-docs`
4. Update mobile `apiConfig.ts` (production URLs)
5. Build and sign mobile release
6. Smoke test all four roles against production API
7. Distribute APK or submit to stores

Rollback

Backend

```
cd /root/backend
git checkout <previous-commit> # if using git on server
npm ci
npx prisma migrate deploy      # only if migration is backward-compatible
pm2 restart guard-tracking-api
```

For breaking schema changes, prepare a down-migration before deploying.

Mobile

Distribute the previous APK/IPA. Mobile clients cache auth tokens; backend must remain compatible with the previous app version during rollout.

Monitoring

Check	Command / URL
API health	<code>GET /api/health</code>
PM2 status	<code>pm2 status</code>
PM2 logs	<code>pm2 logs guard-tracking-api</code>
DB connectivity	<code>npx prisma db execute --stdin</code> (on server)

Related documents

- [CLIENT_HANDOFF.md](#) — delivery overview and go-live checklist
- [ENVIRONMENT_VARIABLES.md](#) — env var reference
- [TROUBLESHOOTING.md](#) — common deployment issues
- [API_REFERENCE.md](#) — endpoint verification